

TECH FORCE: GALACTIC  
ARCHITECTURE

# *DEBUGGER WINDOWS & STACK*



# 8

## LISTA DE FIGURAS

|                                                 |    |
|-------------------------------------------------|----|
| Figura 1 - Interface x32dbg .....               | 7  |
| Figura 2 – Menu de ferramentas .....            | 7  |
| Figura 3 – Menu de ferramentas aproximado ..... | 7  |
| Figura 4 – Abas.....                            | 8  |
| Figura 5 – Aba CPU .....                        | 8  |
| Figura 6 – Aba Graph.....                       | 9  |
| Figura 7 – Aba Breakpoint.....                  | 9  |
| Figura 8 – Aba Memory Map.....                  | 10 |
| Figura 9 – Aba Call Stack.....                  | 10 |
| Figura 10 – Aba References .....                | 11 |
| Figura 11 – Janela principal .....              | 11 |
| Figura 12 – Registradores.....                  | 12 |
| Figura 13 – Stack .....                         | 13 |
| Figura 14 – Stack e Data.....                   | 13 |
| Figura 15 – Dump.....                           | 14 |
| Figura 16 – Crackme Easy.....                   | 14 |
| Figura 17 – Execução do binário.....            | 15 |
| Figura 18 – Abrindo o CrackMe .....             | 15 |
| Figura 19 – Modulo ultimatum .....              | 16 |
| Figura 20 – <i>String</i> references.....       | 16 |
| Figura 21 – Reference.....                      | 17 |
| Figura 22 – <i>Strings</i> suspeitas.....       | 17 |
| Figura 23 – Toggle Breakpoint .....             | 17 |
| Figura 24 – What's the password .....           | 18 |
| Figura 25 – Prompt Ultimatum .....              | 19 |
| Figura 26 – Inserindo a senha.....              | 19 |
| Figura 27 – Input encontrado .....              | 19 |
| Figura 28 – Análise de comparação.....          | 20 |
| Figura 29 – Reiniciar .....                     | 21 |
| Figura 30 – Comparação 4x4.....                 | 21 |
| Figura 31 – Comparação com 52.....              | 21 |
| Figura 32 – Sem jump .....                      | 22 |
| Figura 33 – Comparação da senha.....            | 22 |
| Figura 34 – Teste senha .....                   | 22 |
| Figura 35 – Senha correta.....                  | 23 |

**SUMÁRIO**

|                                           |    |
|-------------------------------------------|----|
| <i>DEBUGGER WINDOWS &amp; STACK</i> ..... | 4  |
| 1 INTRODUÇÃO .....                        | 4  |
| 2 REVISÃO <i>DEBUGGER</i> .....           | 4  |
| 3 X32DBG .....                            | 6  |
| 3.1 Janela Principal .....                | 11 |
| 3.2 Registradores .....                   | 12 |
| 3.3 Stack .....                           | 13 |
| 3.4 Stack e Data .....                    | 13 |
| 3.5 Dump .....                            | 13 |
| 4 CRACK ME 1 - EASY .....                 | 14 |
| CONCLUSÃO .....                           | 23 |
| REFERÊNCIA .....                          | 25 |

## **DEBUGGER WINDOWS & STACK**

### **1 INTRODUÇÃO**

Na era digital contemporânea, a compreensão profunda dos sistemas operacionais e dos programas que neles operam é mais crucial do que nunca. O Windows, sendo um dos sistemas operacionais mais amplamente utilizados, é muitas vezes o alvo de várias formas de malware e ataques cibernéticos. Portanto, é imperativo que os profissionais de TI estejam equipados com as habilidades e ferramentas necessárias para analisar, depurar e, em última análise, fortalecer os programas executados neste sistema operacional.

Este capítulo é dedicado a apresentar duas ferramentas poderosas que são fundamentais para qualquer analista de segurança ou profissional de TI que trabalhe com o Windows, o x32dbg. A ferramenta oferece uma ampla gama de funcionalidades que facilitam a análise e depuração de programas. Além disso, é uma ferramenta essencial para entender o funcionamento interno dos programas e para desenvolver soluções para fortalecê-los contra possíveis ameaças.

Este capítulo servirá como um guia abrangente para instalar, entender e utilizar eficazmente estas ferramentas, bem como aplicar este conhecimento na resolução de 'crackme's, proporcionando assim uma compreensão prática e aplicada das capacidades do x32dbg.

### **2 REVISÃO DEBUGGER**

Um *debugger* ou depurador é uma ferramenta de software usada para testar e depurar outros programas. "Depurar" é o processo de identificar e corrigir erros ou *bugs* em um programa de computador. Um depurador fornece um ambiente onde você pode executar um programa passo a passo, examinar e modificar seu estado (como variáveis, memória, registros de CPU) e identificar problemas.

Aqui estão algumas das principais funções de um depurador:

- **Controle de execução:** O depurador permite que você controle a execução do programa. Você pode executar o programa linha por linha (isso é chamado de "stepping"), executar até um determinado ponto (chamado de "breakpoint"), ou executar até que o programa termine ou encontre um erro.
- **Inspecção de estado:** Você pode inspecionar o estado do programa, incluindo o conteúdo das variáveis, a pilha de chamadas, e o conteúdo da memória.
- **Modificação de estado:** Você pode modificar o estado do programa enquanto ele está sendo executado. Por exemplo, você pode alterar o valor de uma variável, o conteúdo da memória, ou até mesmo o fluxo de execução (por exemplo, pulando sobre uma linha de código).
- **Breakpoints:** Você pode definir *breakpoints* (pontos de parada) em certos pontos do código. Quando o programa atinge um *breakpoint*, ele para de executar, permitindo que você inspecione e modifique seu estado.
- **Tracing:** O depurador pode registrar a execução do programa, incluindo as funções chamadas, os valores das variáveis e as mudanças no estado da memória. Isso pode ser útil para entender o comportamento do programa e identificar problemas.

O funcionamento de um depurador pode variar dependendo da linguagem de programação e do sistema operacional. No entanto, a maioria dos depuradores funciona de maneira semelhante e oferece funcionalidades semelhantes.

Por exemplo, aqui estão alguns passos comuns que você pode seguir ao usar um depurador:

- **Carregar o programa:** Primeiro, você precisa carregar o programa que deseja depurar no depurador. Isso geralmente envolve abrir o código-fonte do programa no depurador.
- **Definir breakpoints:** Em seguida, você pode definir breakpoints no código. Isso são pontos onde você quer que o programa pare de executar para que você possa inspecionar seu estado.
- **Iniciar a execução:** Inicie a execução do programa no depurador. O programa executará normalmente até atingir um *breakpoint* ou terminar.
- **Inspecção e modificação:** Quando o programa atinge um breakpoint, você pode inspecionar e modificar seu estado. Você também pode executar o programa linha por linha ou até o próximo breakpoint.

- **Repetir:** Repita os passos 3 e 4 até ter identificado e corrigido todos os erros no programa.
- **Finalizar:** Uma vez que você tenha corrigido todos os erros, você pode finalizar a execução do programa.

No tópico a seguir iremos usar o x32dbg para entender como funciona o debbuging de programas.

### 3 X32DBG

O x32dbg é uma parte do projeto x64dbg, um depurador *open source* (código aberto) para sistemas Windows de 32 e 64 bits. O projeto é composto por três componentes: o x32dbg para programas de 32 bits, o x64dbg para programas de 64 bits e o x96dbg para ambos.

O x32dbg é uma ferramenta poderosa para engenharia reversa de aplicativos de 32 bits. Ele possui uma interface gráfica do usuário (GUI) intuitiva e personalizável, que facilita a navegação e a execução de tarefas de depuração. Além disso, tem um mecanismo de script embutido que permite a automação de várias tarefas de depuração e engenharia reversa, usando uma linguagem de script especial chamada "x64dbg script", que é semelhante ao Assembly.

O x32dbg também fornece várias ferramentas para análise de código, como visualização de gráficos de controle de fluxo, busca por padrões de código e análise de referências cruzadas. Os usuários podem manipular dados em tempo real durante a depuração, modificando registros, pilhas e memória diretamente da interface do depurador. Além disso, o x32dbg suporta plugins, permitindo que a comunidade crie extensões para adicionar funcionalidades ao depurador. Por fim, é uma aplicação portátil, o que significa que não requer instalação e pode ser executada a partir de um dispositivo USB ou de uma pasta no sistema de arquivos.

O x32dbg pode ser baixado do link <https://x64dbg.com>.

Vamos ver agora como é a interface do x32dbg.

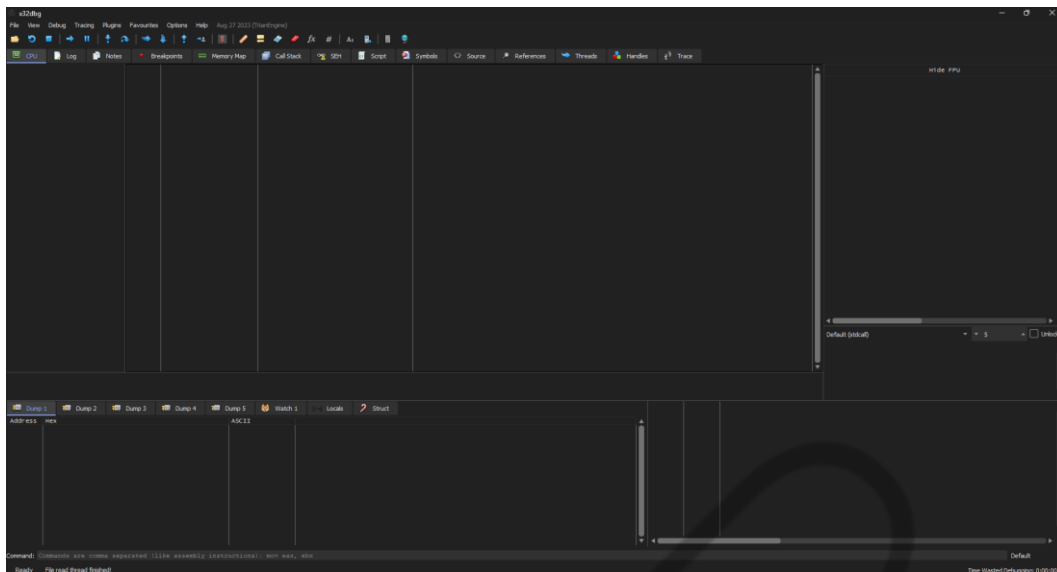


Figura 1 - Interface x32dbg  
Fonte: Elaborada pelo autor (2023)

Essa é a interface do x32dbg, cada região dessa interface irá apresentar dados específicos que são importantes quando estivermos debugando um arquivo. Vamos esmiuçar cada uma.

O primeiro elemento que é importante ressaltar é o Menu de ferramentas principal

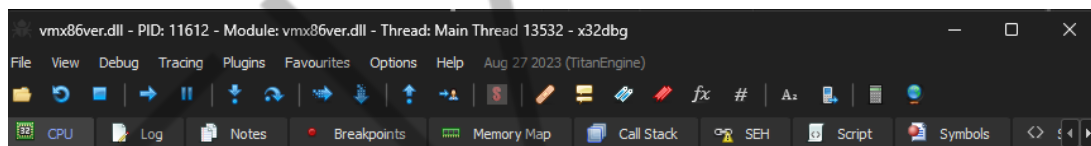


Figura 2 – Menu de ferramentas  
Fonte: Elaborada pelo autor (2023)

A parte externa da janela irá fornecer para nós o nome do arquivo que está sendo analisado, nesse caso uma dll (vmx86ver.dll), o PID do processo que foi dado pelo sistema operacional e o módulo que estamos analisando (vmx86ver.dll).

Os ícones na barra de ferramentas são essenciais para explorar o código assembly. Os principais para um iniciante são:



Figura 3 – Menu de ferramentas aproximado  
Fonte: Elaborada pelo autor (2023)

- **Pasta:** Usada para abrir o arquivo que será analisado;
- **Seta circular:** Usada para retornar ao início do programa;
- **Ícone de parada:** Usado para parar o programa se estiver executando no x32dbg;
- **Seta:** Irá executar o programa;
- **Pause:** Irá pausar o programa.

Porém, os que são mais utilizados pelo analista são os últimos dois. O primeiro, Step Into, é usado para que possamos entrar dentro de uma função para ver o que ela está fazendo. Já o Step Over, é muito usado para pular alguma função que não é de interesse.

A linha logo abaixo dos ícones, apresenta algumas abas muito importantes.



Figura 4 – Abas

Fonte: Elaborada pelo autor (2023)

A aba CPU apresenta o código em assembly do início ao final do programa:

|          |                    |                                                                 |                  |
|----------|--------------------|-----------------------------------------------------------------|------------------|
| 772B7910 | 55                 | push ebp                                                        |                  |
| 772B7911 | 8BEC               | mov ebp,esp                                                     |                  |
| 772B7913 | 83E4 F8            | and esp,FFFFFFF8                                                |                  |
| 772B7916 | 81EC 70010000      | sub esp,170                                                     |                  |
| 772B791C | A1 70233377        | mov eax,dword ptr ds:[77332370]                                 |                  |
| 772B7921 | 33C4               | xor eax,esp                                                     |                  |
| 772B7923 | 898424 6C010000    | mov dword ptr ss:[esp+16C],eax                                  |                  |
| 772B792A | 56                 | push esi                                                        |                  |
| 772B792B | 8B35 F8013377      | mov esi,dword ptr ds:[773301F8]                                 |                  |
| 772B7931 | 57                 | push edi                                                        |                  |
| 772B7932 | 6A 16              | push 16                                                         |                  |
| 772B7934 | 58                 | pop eax                                                         |                  |
| 772B7935 | 66:894424 10       | mov word ptr ss:[esp+10],ax                                     |                  |
| 772B793A | 8BF9               | mov edi,ecx                                                     |                  |
| 772B793C | 6A 18              | push 18                                                         |                  |
| 772B793E | 58                 | pop eax                                                         |                  |
| 772B793F | 66:894424 12       | mov word ptr ss:[esp+12],ax                                     |                  |
| 772B7944 | 8D4424 70          | lea eax,dword ptr ss:[esp+70]                                   |                  |
| 772B7948 | 894424 6C          | mov dword ptr ss:[esp+6C],eax                                   |                  |
| 772B794C | 33C0               | xor eax,eax                                                     |                  |
| 772B794E | C74424 14 68712077 | mov dword ptr ss:[esp+14],ntdll.7720716877207168:L"apphelp.dll" |                  |
| 772B7956 | C74424 68 00000001 | mov dword ptr ss:[esp+68],1000000                               |                  |
| 772B795E | 66:894424 70       | mov word ptr ss:[esp+70],ax                                     |                  |
| 772B7963 | 85F6               | test esi,esi                                                    |                  |
| 772B7965 | 74 29              | je ntdll.772B7990                                               |                  |
| 772B7967 | 8B15 3003FE7F      | mov edx,dword ptr ds:[7FFE0330]                                 | 7FFE0330: "D3ov" |
| 772B796D | 8BC2               | mov eax,edx                                                     |                  |
| 772B796F | 6A 20              | push 20                                                         |                  |

Figura 5 – Aba CPU

Fonte: Elaborada pelo autor (2023)

Já a aba Graph, fornece uma forma de visualizar o programa para que o usuário consiga ver qual função é chamada por outra e ter uma ideia do fluxo de execução do programa:



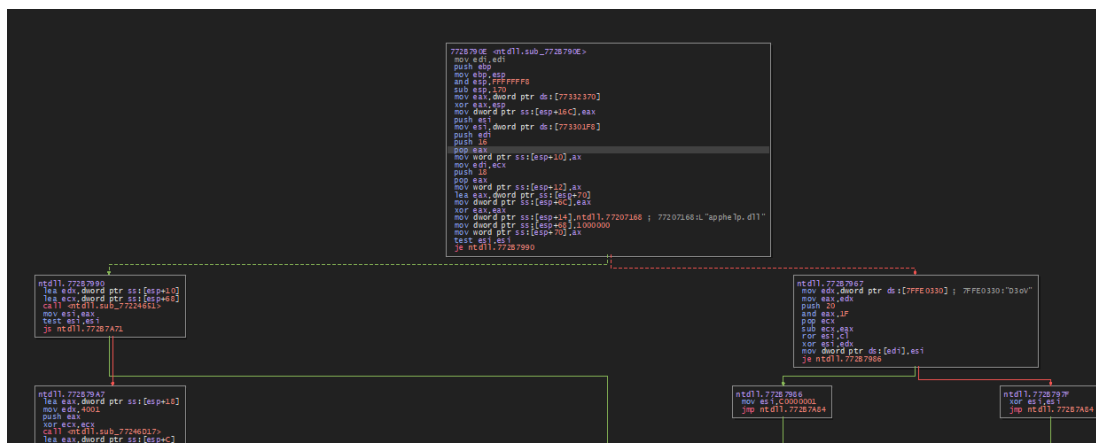


Figura 6 – Aba Graph  
Fonte: Elaborada pelo autor (2023)

Por sua vez, Breakpoints, possibilita a visualização de todos os breakpoints que foram inseridos no *debugger* para que ele pare naquele determinado ponto do código, que normalmente é um ponto de interesse do analista.

A aba Breakpoints é usada para busca por instruções ou *strings* que pegaram o interesse do analista e um **breakpoint** foi inserido naquele endereço. Breakpoints nada mais são que interrupções na execução do programa que fazem com que a sequência de execução seja pausada naquele ponto. Isso é muito útil quando queremos marcar pontos de interesse ou quando queremos realizar uma análise mais a fundo em um determinado ponto do código.

| Type     | Address          | Module/Label/Exception | State    | Disassembly                       | Hits | Summary               |
|----------|------------------|------------------------|----------|-----------------------------------|------|-----------------------|
| Software | 0000000100401000 | <prova.exe.EntryPoint> | One-time | sub rsp, 28                       | 0    | entry breakpoint      |
|          | 0000000100401168 | prova.exe              | Enabled  | lea rax, qword ptr ds:[100404000] | 0    |                       |
|          | 000000010040122C | prova.exe              | Enabled  | lea rax, qword ptr ds:[100404161] | 0    |                       |
|          | 0000000100401248 | prova.exe              | Enabled  | lea rcx, qword ptr ds:[100404174] | 0    |                       |
|          | 00000001004012D3 | prova.exe              | Enabled  | lea r8, qword ptr ds:[100404180]  | 0    |                       |
|          | 0000000100401439 | prova.exe              | Enabled  | lea rcx, qword ptr ds:[100404170] | 0    |                       |
|          | 0000000100401D6A | prova.exe              | Enabled  | lea r8, qword ptr ds:[100404208]  | 0    |                       |
|          | 00007FB68024F98  | ntdll.dll              | Enabled  | call <ntdll.ZwTestAlert>          | 0    | Faz muitas coisas     |
|          | 00007FB68083C25  | ntdll.dll              | Enabled  | call <ntdll.7FB6801BF3>           | 0    | inicializa o programa |

Figura 7 – Aba Breakpoint  
Fonte: Elaborada pelo autor (2023)

A próxima Janela é a do Memory Map, nela podemos ver dados que estão armazenados na memória e o endereço no qual os dados residem. Um exemplo, a seção `.text` do programa analisado está localizado no endereço 0000000100401000.

| Address          | Size               | Info                                | Content | Type | Protection | Initial |
|------------------|--------------------|-------------------------------------|---------|------|------------|---------|
| 0000000000010000 | 0000000000010000   |                                     |         | MAP  | -RW--      | -RW--   |
| 0000000000020000 | 0000000000010000   |                                     |         | MAP  | -R---      | -R---   |
| 0000000000030000 | 0000000000010000   |                                     |         | MAP  | -R---      | -R---   |
| 0000000000040000 | 0000000000010000   |                                     |         | MAP  | -R---      | -R---   |
| 0000000000050000 | 0000000000010000   |                                     |         | MAP  | -R---      | -R---   |
| 0000000000060000 | 0000000000010000   |                                     |         | MAP  | -R---      | -R---   |
| 0000000000070000 | 0000000000020000   |                                     |         | PRV  | -RW--      | -RW--   |
| 0000000000080000 | 00000000000C9000   | \Device\HarddiskVolume3\windows\... |         | MAP  | -R---      | -R---   |
| 0000000000090000 | 00000000000001000  |                                     |         | MAP  | -R---      | -R---   |
| 00000000000A0000 | 00000000000001000  |                                     |         | MAP  | -R---      | -R---   |
| 00000000000B0000 | 00000000000001000  |                                     |         | MAP  | -R---      | -R---   |
| 00000000000C0000 | 00000000000001000  |                                     |         | PRV  | -RW--      | -RW--   |
| 00000000000D0000 | 00000000000001000  | Reserved                            |         | PRV  | -RW--      | -RW--   |
| 00000000000E0000 | 0000000000009000   | PEB                                 |         | PRV  | -RW--      | -RW--   |
| 00000000000F0000 | 00000000000000000  | Reserved (0000000000200000)         |         | PRV  | -RW--      | -RW--   |
| 0000000000100000 | 000000000001FA000  | Reserved                            |         | PRV  | -RW--      | -RW--   |
| 0000000000110000 | 00000000000000000  | Thread 18c0 stack                   |         | PRV  | -RW-G      | -RW--   |
| 0000000000120000 | 000000000000017000 | Reserved (0000000000780000)         |         | PRV  | -RW--      | -RW--   |
| 0000000000130000 | 00000000000000000  | Reserved                            |         | PRV  | -RW--      | -RW--   |
| 0000000000140000 | 00000000000000000  | Thread 196C stack                   |         | PRV  | -RW-G      | -RW--   |
| 0000000000150000 | 00000000000000000  | Reserved                            |         | PRV  | -RW--      | -RW--   |
| 0000000000160000 | 00000000000000000  | Thread 1968 stack                   |         | PRV  | -RW-G      | -RW--   |
| 0000000000170000 | 00000000000000000  | Reserved                            |         | PRV  | -RW--      | -RW--   |
| 0000000000180000 | 00000000000000000  | Thread 1964 stack                   |         | PRV  | -RW-G      | -RW--   |
| 0000000000190000 | 00000000000000000  | KUSER_SHARED_DATA                   |         | PRV  | -R---      | -R---   |
| 00000000001A0000 | 00000000000000000  | prova.exe                           |         | IMG  | -R---      | ERWC-   |
| 00000000001B0000 | 00000000000000000  | Executable code                     |         | IMG  | ERWC-      | ERWC-   |
| 00000000001C0000 | 00000000000000000  | data                                |         | IMG  | -RWC-      | ERWC-   |
| 00000000001D0000 | 00000000000000000  | Read-only initialized data          |         | IMG  | -R---      | ERWC-   |
| 00000000001E0000 | 00000000000000000  | Exception information               |         | IMG  | -R---      | ERWC-   |
| 00000000001F0000 | 00000000000000000  | uninitialized data                  |         | IMG  | -RWC-      | ERWC-   |
| 0000000000200000 | 00000000000000000  | Import tables                       |         | IMG  | -RW-       | ERWC-   |
| 0000000000210000 | 00000000000000000  | Resources                           |         | IMG  | -RW-       | ERWC-   |
| 0000000000220000 | 00000000000000000  | "/4"                                |         | IMG  | -R---      | ERWC-   |
| 0000000000230000 | 00000000000000000  | "/19"                               |         | IMG  | -R---      | ERWC-   |
| 0000000000240000 | 00000000000000000  | "/31"                               |         | IMG  | -R---      | ERWC-   |
| 0000000000250000 | 00000000000000000  | "/45"                               |         | IMG  | -R---      | ERWC-   |
| 0000000000260000 | 00000000000000000  | "/57"                               |         | IMG  | -R---      | ERWC-   |
| 0000000000270000 | 00000000000000000  | "/70"                               |         | IMG  | -R---      | ERWC-   |
| 0000000000280000 | 00000000000000000  | "/81"                               |         | IMG  | -R---      | ERWC-   |

Figura 8 – Aba Memory Map  
Fonte: Elaborada pelo autor (2023)

O Call Stack é útil quando um breakpoint tiver sido inserido em uma função específica do Windows. Então, por exemplo se eu tivesse inserido um input na função CreateDirectoryW, toda vez que o programa fosse criar um diretório no sistema, o breakpoint será atingido e o x64dbg irá parar nessa função. Usando o call Stack conseguimos ver qual o código escrito pelo programa que está utilizando o CreateDirectoryW e identificar onde o diretório está sendo criado.

| Thread | Address          | To               | From              | Size | Comment                      |
|--------|------------------|------------------|-------------------|------|------------------------------|
| 6336   | 00000000005FFF28 | 00007FFB68647034 | 0000000100401000  | 30   | prova.EntryPoint             |
| 6508   | 00000000005FFF58 | 00007FFB68D02651 | 00007FFB68647034  | 80   | kernel32.00007FFB68647034    |
| 6500   | 0000000000A7FC28 | 00007FFB68D02C7  | 00007FFB68D0507C4 | 300  | ntdll.sub_7FFB68C89D50+96A74 |
| 6504   | 0000000000A7FF28 | 00007FFB68D02651 | 00007FFB68647034  | 80   | ntdll.sub_7FFB68C89D50+49077 |
| 6504   | 0000000000E7FC28 | 00007FFB68D02C7  | 00007FFB68D0507C4 | 300  | ntdll.sub_7FFB68C89D50+96A74 |
| 6504   | 0000000000E7FF28 | 00007FFB68D02651 | 00007FFB68647034  | 80   | ntdll.sub_7FFB68C89D50+49077 |
| 6504   | 0000000000E7FFD8 | 00007FFB68D02651 | 00007FFB68647034  | 80   | kernel32.00007FFB68647034    |
| 6504   | 0000000000C7FC28 | 00007FFB68D02C7  | 00007FFB68D0507C4 | 300  | ntdll.sub_7FFB68C89D50+96A74 |
| 6504   | 0000000000C7FF28 | 00007FFB68D02651 | 00007FFB68647034  | 80   | ntdll.sub_7FFB68C89D50+49077 |
| 6504   | 0000000000C7FFD8 | 0000000000000000 | 00007FFB68D02651  | 80   | ntdll.sub_7FFB68C89D50+48901 |

Figura 9 – Aba Call Stack  
Fonte: Elaborada pelo autor (2023)

Por fim, a última aba que é interessante conhecer é a References. Nela quando buscamos por referências de *string* no módulo que está sendo debugado podemos encontrar padrões de texto dentro do código para que possamos encontrar pontos de interesse do qual partir. Na imagem abaixo temos alguns exemplos de *strings* interessantes:

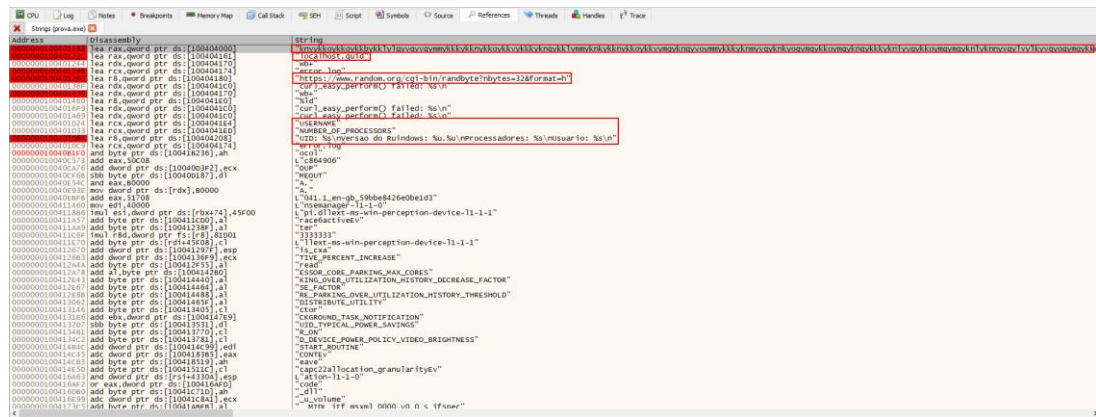


Figura 10 – Aba References  
Fonte: Elaborada pelo autor (2023)

### 3.1 Janela Principal

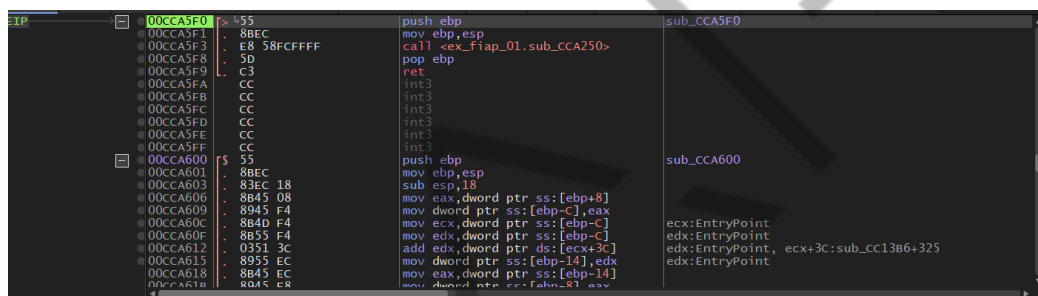


Figura 11 – Janela principal  
Fonte: Elaborada pelo autor (2023)

A janela principal do xdbg contém o conteúdo da aba selecionada, por padrão esta é a aba CPU que contém as instruções de assembly. A primeira coluna contém o EIP, que é o registrador de instruções. Ele aponta para o próximo código que irá ser executado.

A segunda coluna contém os endereços de dentro do binário onde as instruções residem. A terceira coluna é a representação hexadecimal das instruções na coluna 4, que por sua vez é onde as instruções em assembly estão localizadas. No caso acima a coluna 4 possui a instrução “push ebp”.

A quinta coluna contém dados populado pelo xdbg ou notas que foram adicionadas pelo analista.

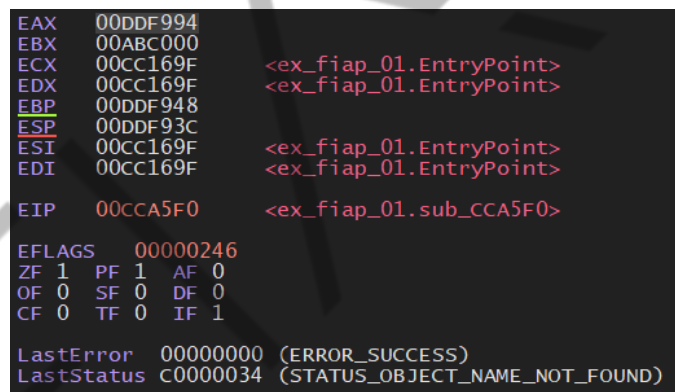
A coluna que possui as instruções é a que mais vamos utilizar e a que devemos prestar mais atenção dentro dessa janela.

Para facilitar vamos fazer uma pequena revisão de algumas instruções básicas:

- **PUSH:** Envia um valor para Stack;
- **POP:** Retira um valor da Stack;
- **CALL:** Executa uma função;
- **RET:** Retorna um valor de uma função completa;
- **JMP:** Salta para um determinado endereço;
- **CMP:** Compara dois valores;
- **MOV:** Movimenta dados de uma localização para outra;
- **ADD:** Soma dois valores;
- **SUB:** Subtrai dois valores.

### 3.2 Registradores

A região logo a direita da janela principal mostra os registradores e os valores que se encontram dentro deles:



```
EAX 00DDF994
EBX 00ABC000
ECX 00CC169F <ex_fiap_01.EntryPoint>
EDX 00CC169F <ex_fiap_01.EntryPoint>
EBP 00DDF948
ESP 00DDF93C
ESI 00CC169F <ex_fiap_01.EntryPoint>
EDI 00CC169F <ex_fiap_01.EntryPoint>
EIP 00CCA5F0 <ex_fiap_01.sub_CCA5F0>

EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C0000034 (STATUS_OBJECT_NAME_NOT_FOUND)
```

Figura 12 – Registradores  
Fonte: Elaborada pelo autor (2023)

Somente para lembrar sobre registradores:

- **EAX:** Usado para adições, multiplicações e retorno de valores das funções;
- **EBX:** Registrador genérico, usado para várias operações;
- **ECX:** Usado para contadores;
- **EDX:** Registrador genérico, usado para várias operações;
- **EBP:** Usado para referenciar argumentos e variáveis locais;
- **ESP:** Aponta para o último argumento da Stack;

- **ESI/EDI:** Usados para transferência de dados entre localizações na memória;
- **EIP:** Registrador que aponta para próxima instrução.

### 3.3 Stack

A janela logo abaixo dos registradores mostra a pilha ou Stack. As informações contidas aqui são parâmetros que foram enviadas para a Stack.

```
1: [esp+4] 00ABC000
2: [esp+8] 76A4FA10 <kernel32.BaseThreadInitThunk>
3: [esp+C] 00DDF9A4
4: [esp+10] 77227A9E ntdll.77227A9E
5: [esp+14] 00ABC000
```

Figura 13 – Stack  
Fonte: Elaborada pelo autor (2023)

### 3.4 Stack e Data

A região logo abaixo da Stack contém a Stack e dados que foram enviados para ela e os endereços de memória para os quais eles foram mapeados.

```
00DDF93C 76A4FA29 return to kernel32.76A4FA29 from ???
00DDF940 00ABC000
00DDF944 76A4FA10 kernel32.76A4FA10
00DDF948 00DDF9A4
00DDF94C 77227A9E return to ntdll.77227A9E from ???
00DDF950 00ABC000
00DDF954 2D5C7599
00DDF958 00000000
00DDF95C 00000000
00DDF960 00ABC000
00DDF964 00000000
00DDF968 00000000
```

Figura 14 – Stack e Data  
Fonte: Elaborada pelo autor (2023)

### 3.5 Dump

A maior região inferior do xdbg contém o dump dos dados. Essa janela permite que o analista veja os dados que estão armazenados no registrador ou qual dado reside em um determinado endereço.

Quando selecionamos um registrador e queremos ver que instrução tem dentro dele ou mesmo quando operações de ponteiros fornecem endereços sem fornecer a

informação dentro deles podemos clicar com o botão direito naquela instrução e fazer o dump dela que será mostrado nessa região.

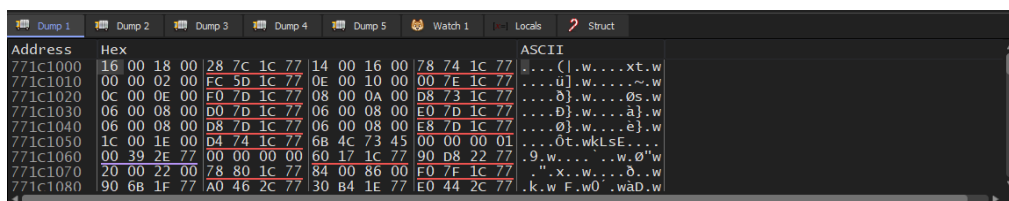


Figura 15 – Dump  
Fonte: Elaborada pelo autor (2023)

Uma vez que entendemos todas as partes do x32dbg, vamos agora fazer um exercício de fixação.

## 4 CRACK ME 1 - EASY

Vamos pegar de exemplo um crackme simples do site crackme.org. Esse site possui diferentes níveis de binários para treinarmos a análise de executáveis. O que vamos usar para esse exemplo chama-se Ultimatum CrackMe foi desenvolvido pelo programador que assina como RedXen. Ele pode ser baixado do seguinte link <https://crackmes.one/crackme/61ae4e9733c5d413767c9d69>.

É um binário de Windows x86 de dificuldade 1.3 escrito em C/C++. O primeiro passo é extrair o binário e convertê-lo para executável.

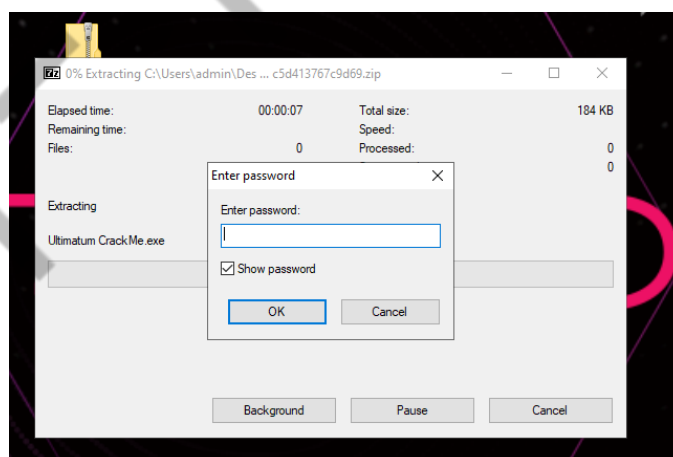


Figura 16 – Crackme Easy  
Fonte: Elaborada pelo autor (2023)



Feito isso renomeamos o arquivo colocando a extensão .exe. Uma vez convertido, vamos executá-lo para ver o que o binário faz:

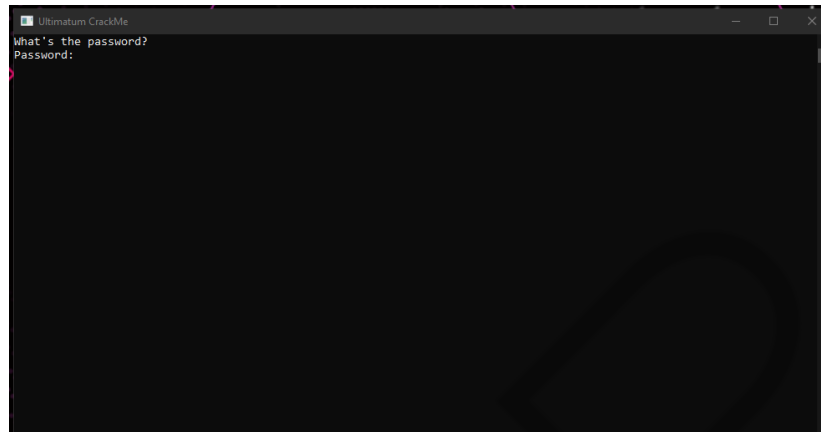


Figura 17 – Execução do binário  
Fonte: Elaborada pelo autor (2023)

Parece ser um programa que pede para inserirmos uma senha. Fornece a frase What's the password e o campo Password: para que possamos inserir credenciais. Vamos guardar essa informação por hora e importar ele no x32dbg através do ícone da pasta, ícone 1 na imagem abaixo.

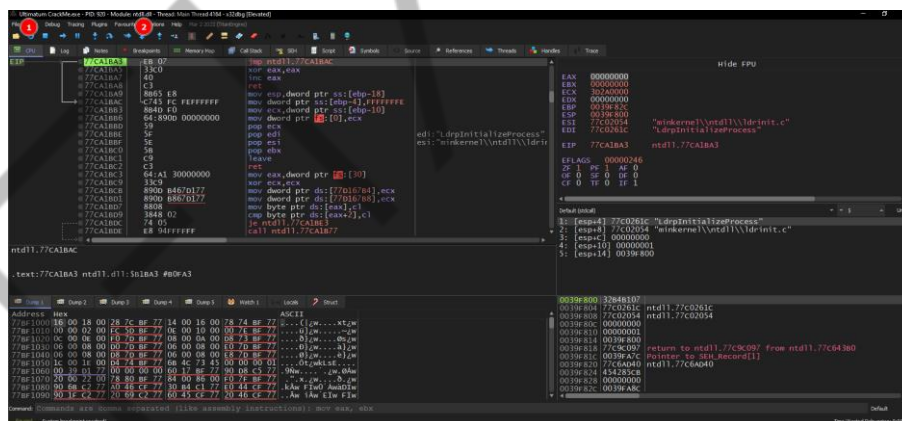


Figura 18 – Abrindo o CrackMe  
Fonte: Elaborada pelo autor (2023)

Note que ao abrirmos o executável o módulo do kernel é primeiramente carregado. Ele irá iniciar os processos e ambiente necessário para que o binário possa operar. Dessa forma, precisamos continuar o código até que cheguemos de fato no programa que queremos analisar. Para fazer isso basta clicarmos na seta run (ou F9) se preferir. O *debugger* irá parar no *entry point* (ponto de entrada) do programa Ultimatum que é seu início. Por padrão um breakpoint é inserido pelo depurador no *entry point* do programa.

Bem, uma vez que estamos com o programa carregado - podemos ver isso através dos detalhes da janela do *debugger* - precisamos encontrar pontos de interesse para que possamos analisar.

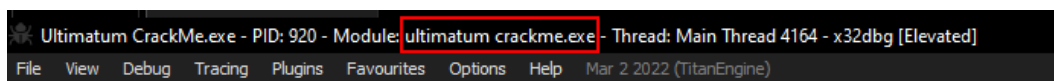


Figura 19 – Módulo ultimatum  
Fonte: Elaborada pelo autor (2023)

Para isso vamos buscar as referências do módulo clicando com o botão direito na região do dissassembler (aba CPU) e selecionando Search for > Current Module > *String reference*:

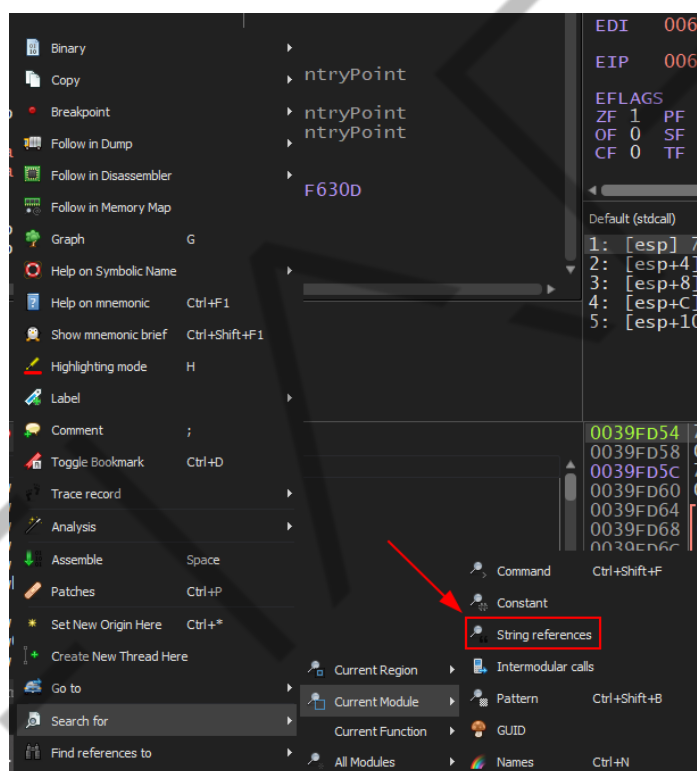


Figura 20 – *String references*  
Fonte: Elaborada pelo autor (2023)

A aba de Reference abrirá e poderemos ver todas as *strings* contidas no programa:



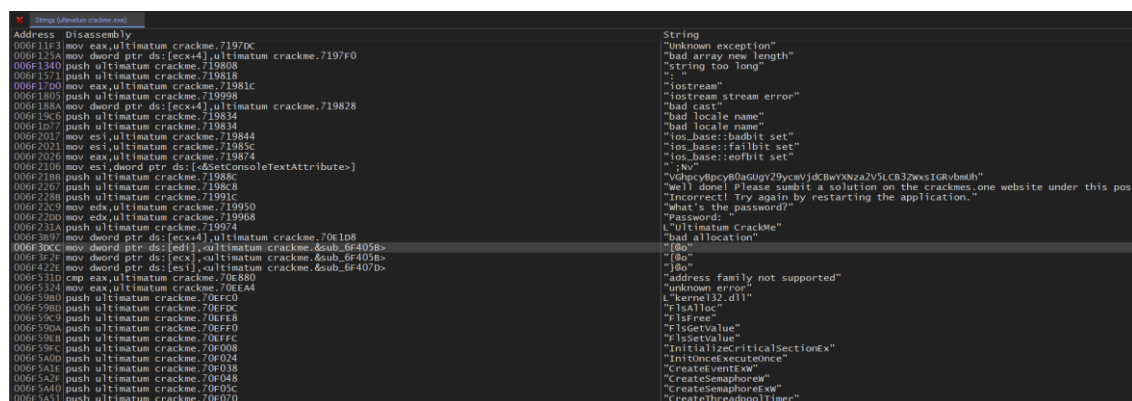


Figura 21 – Reference

Fonte: Elaborada pelo autor (2023)

Veja se consegue identificar na imagem *strings* que parecem interessantes. Se pensou nas *strings* da imagem abaixo, muito bem:



Figura 22 – Strings suspeitas

Fonte: Elaborada pelo autor (2023)

Mas não se preocupe se não percebeu elas, afinal estamos aqui para aprender. Onde vimos essas *strings* antes?! Quando executamos o programa!!

Agora entramos nas etapas de testar hipóteses e verificar como o programa se comporta. Mas antes vamos colocar um breakpoint nos pontos de interesse. Para isso basta clicarmos com o botão direito na linha que queremos inserir um breakpoint e selecionar Toggle breakpoint, ou pressionar F2.



Figura 23 – Toggle Breakpoint

Fonte: Elaborada pelo autor (2023)

Os breakpoints inseridos terão seus endereços em vermelho. Também podemos vê-los na aba Breakpoints.

Inseridos os breakpoints, vamos testar uma hipótese. Se o programa pede uma senha ele precisará comparar essa senha com alguma coisa para saber se está certa ou errado. Dessa forma, podemos tentar achar essa comparação e encontrar o valor contido no programa com o qual nosso input é comparado.

Primeiro passo é irmos até o ponto onde inserimos a senha. Para isso voltamos a aba CPU e pressionamos F9.

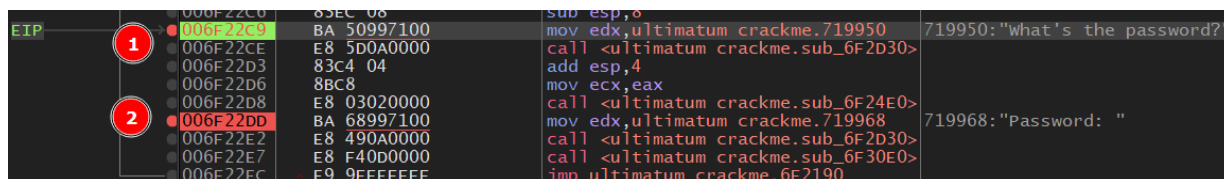


Figura 24 – What's the password  
Fonte: Elaborada pelo autor (2023)

Olha só onde paramos! Exatamente no ponto em que o programa pergunta qual o *password*. Perceba como esse processo é muito mais rápido que ir passando cada uma das instruções até chegar nesse ponto. Esse é um dos motivos pelos quais tentamos sempre buscar pontos de interesse no programa para que possamos ter um apoio na hora da análise.

Agora podemos ir de instrução em instrução analisando onde que o programa pede para inserirmos a senha. Para fazer esse passo a passo, selecionamos a seta step over (F8). Logo na sequência o EIP aponta para uma *call*. Sabemos que uma *call* é uma função que é chamada pelo programa que irá retornar algum valor, porém não sabemos se ela de fato é interessante para nossa análise ou não. Como nosso objetivo é encontrar o ponto no qual inserimos a senha vamos apertar novamente F8 para passarmos pela função sem entrar nela. Mais alguns F8s e chegamos na *string* "Password:".

Lembrando de quando executamos o programa fora do *debugger*, essa *string* era mostrada antes de inserirmos a senha. Dessa forma, uma das próximas instruções deve ser o *input* da senha. A primeira *call* logo após a linha *Password* mostra no terminal a linha "Password: "(Sempre que o programa é aberto no *debugger* ele também é iniciado no sistema. Assim clicando na janela ao lado do *debugger* você irá ver que o prompt de comando mostra o que o programa está executando naquele momento).

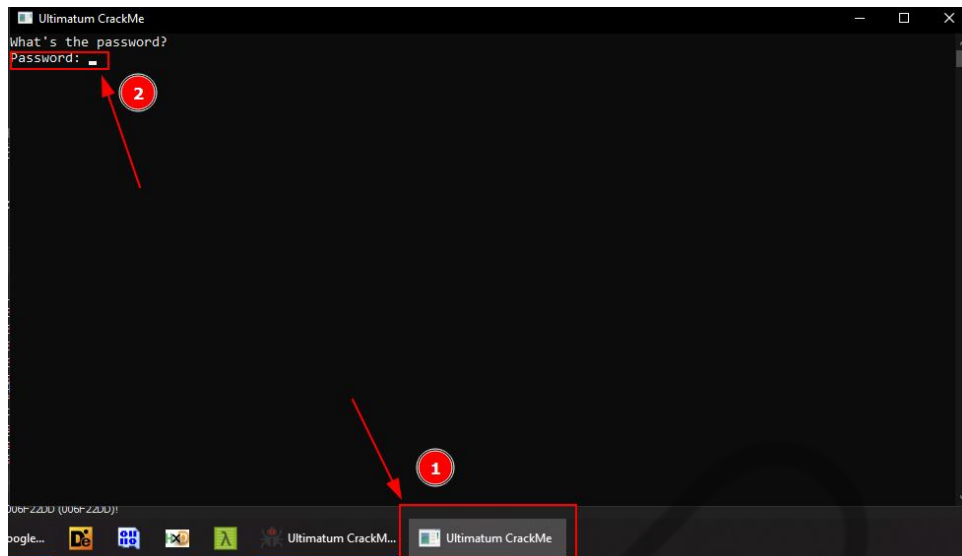


Figura 25 – Prompt Ultimatum  
Fonte: Elaborada pelo autor (2023)

Mais um F8 e o programa para na segunda *call*. Podemos apertar F8 de novo, porém o programa não parece fazer nada. Por quê?! Simples, ele está esperando que digitemos a senha. Então encontramos o local no qual inserimos a senha.

Vamos colocar algo simples para que possamos mapear no programa e seguir nossa senha até o ponto no qual ela é comparada. Vou inserir 1234567890 o que equivale a “A” em hexadecimal (lembre-se que todos os dados são mostrados em hexadecimal exceto se for uma *string* em texto plano).

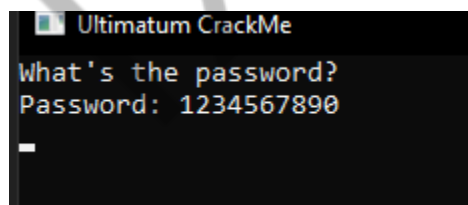


Figura 26 – Inserindo a senha  
Fonte: Elaborada pelo autor (2023)

Mais um F8 e o EIP aponta para um JMP. Bom por enquanto não achamos nenhum indício da nossa senha inserida. Continuemos nossa análise apertando F8 e buscando nos registradores algum indício da nossa senha inserida.

Alguns “Steps” mais tarde encontramos as seguintes linhas:

|          |                  |                                     |                                 |
|----------|------------------|-------------------------------------|---------------------------------|
| 006F21B8 | 68 8C987100      | push ultimatum crackme.71988C       | 71988C: "VghpcyBpcyB0agUgY29ycm |
| 006F21C0 | 8D4D D8          | lea ecx,dword ptr ss:[ebp-28]       |                                 |
| 006F21C3 | C745 E8 00000000 | mov dword ptr ss:[ebp-18],0         |                                 |
| 006F21CA | C745 EC 0F000000 | mov dword ptr ss:[ebp-14],F         | [ebp-14]:sub_70D55D             |
| 006F21D1 | C645 D8 00       | mov byte ptr ss:[ebp-28],0          |                                 |
| 006F21D5 | E8 36040000      | call <ultimatum crackme.sub_6F2610> |                                 |
| 006F21DA | C745 FC 00000000 | mov dword ptr ss:[ebp-4],0          |                                 |
| 006F21E1 | 8D4D D8          | lea ecx,dword ptr ss:[ebp-28]       |                                 |
| 006F21E4 | 837D EC 10       | cmp dword ptr ss:[ebp-14],10        | [ebp-14]:sub_70D55D             |
| 006F21E8 | BA 78CA7100      | mov edx,ultimatum crackme.71CA78    | 71CA78: "1234567890"            |

Figura 27 – Input encontrado

Fonte: Elaborada pelo autor (2023)

Encontramos a *string* estranha com diferentes caracteres e nossa senha (última linha da imagem). Nesse caso, parece que nossa senha está sendo movimentada para o registrador EDX. Se passarmos pela linha com F8 e conferirmos o registrador EDX é exatamente isso que ocorre, porém nada de interessante acontece além dessa movimentação. No entanto, um pouco mais abaixo mais uma movimentação é feita e dessa vez nossa senha passa para o registrador ESI que é comparado ao valor contido no endereço [ebp-18]. Se clicarmos com o botão direito sobre essa linha e selecionarmos “Follow dump” podemos ver o que ele contém:

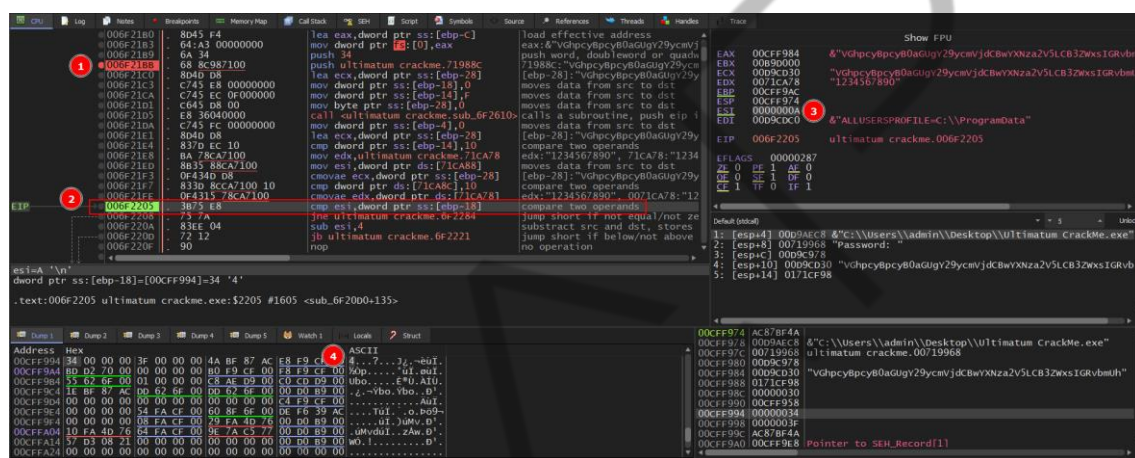


Figura 28 – Análise de comparação  
Fonte: Elaborada pelo autor (2023)

O número 1 é o local onde encontramos a *string* esquisita de caracteres “aleatórios”. Já o número 2 é a linha da comparação entre nossa senha e o endereço [ebp-18]. 3 indica o ESI contendo o tamanho da nossa senha (10 em decimal, ou A em hexadecimal). Por fim, 4 é o dump que fizemos do endereço [ebp-18]. Verificando essas informações descobrimos que o programa está comparando o tamanho da nossa senha com o valor 4. Será que isso significa que a senha tem 4 caracteres de tamanho?! Não sabemos, mas pode ser um indicador! Vamos colocar um breakpoint nesse ponto e seguir.

Vamos continuar com nossa análise por hora. Mais alguns F8s o programa faz um jne (jump quando os valores são diferentes, 4 não é igual a A, certo?!) e nos fornece a saída de que a senha está errada.

Bom, hora de reiniciarmos o programa e testarmos uma senha de 4 caracteres e ver se isso muda algo no comportamento do programa. Para reiniciar o programa basta selecionar a seta circular na aba de ferramentas:

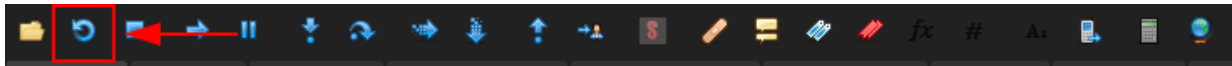


Figura 29 – Reiniciar  
Fonte: Elaborada pelo autor (2023)

Voltamos para o mesmo ponto inserimos uma senha de 4 caracteres e chegamos novamente à comparação:

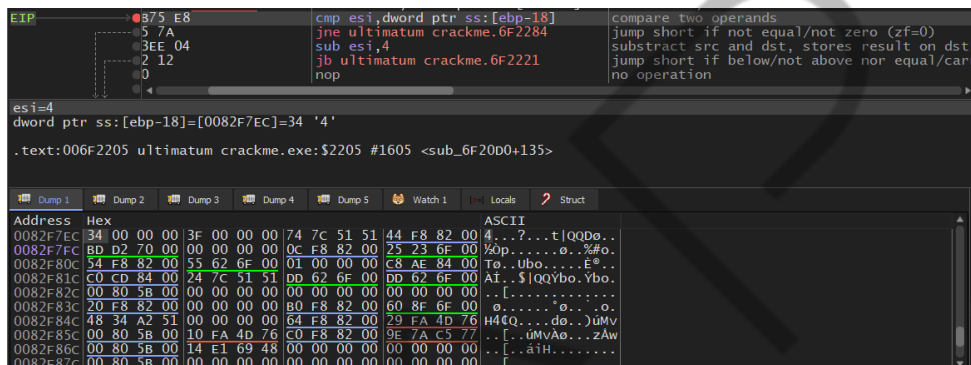


Figura 30 – Comparação 4x4  
Fonte: Elaborada pelo autor (2023)

Aqui o programa compara 4 com 4 e faz o salto. Ué?! Não era para ele não fazer o salto?! O que ocorreu?!

Analisando mais de perto, o programa não está de fato comparando o tamanho da nossa *string* com o caractere 4, porém está verificando se ele tem 34 caracteres de tamanho. No entanto, cuidado! Lembre-se que 34 está em hexadecimal e não em decimal. Logo, a comparação é validando se o tamanho da senha é de 52 caracteres (34 em hexa é 52 em decimal).

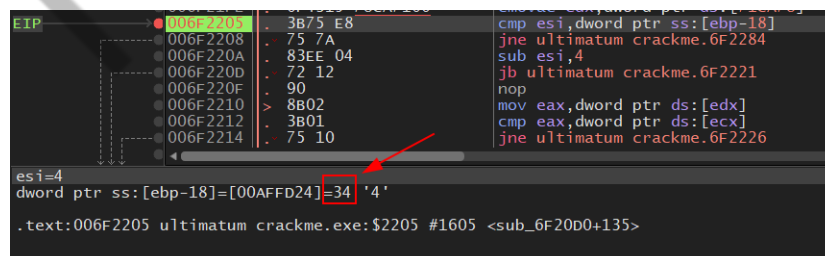


Figura 31 – Comparação com 52  
Fonte: Elaborada pelo autor (2023)

Logo abaixo da região da aba CPU podemos ver um pequeno retângulo que fornece para nós a tradução dos endereços no qual EIP está apontando. Ou seja, o endereço [ebp-18] que estávamos vendo. Veja como o valor dentro dele é 34 que é



então traduzido para '4'. Esse 4 nada mais é do que o *debugger* traduzindo para ASCII o valor 34. Porém, a comparação é feita entre números e não entre *strings*.

Bem, reiniciemos o programa e agora inserimos 52 caracteres como senha para vermos o que acontece:

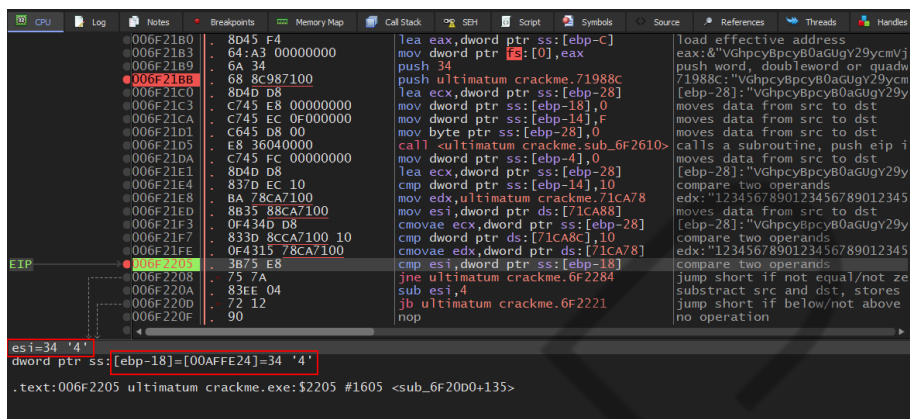


Figura 32 – Sem jump  
Fonte: Elaborada pelo autor (2023)

Agora sim! Continuamos sem o jump e seguimos no programa.

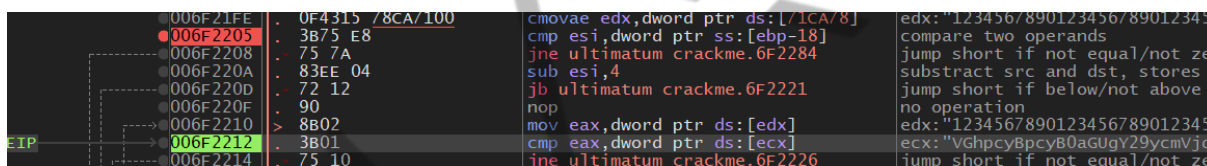


Figura 33 – Comparação da senha  
Fonte: Elaborada pelo autor (2023)

Um pouco mais abaixo vemos que nossa senha é movida para o registrador EAX e então comparado com a *string* esquisita que encontramos lá atrás no programa. Será que encontramos a senha?!

Como a comparação não é verdadeira o salto jne após a comparação ocorre e recebemos o resultado que a senha estava errada.

Vamos testar agora a senha usando a *string* esquisita.

Copiando-a do endereço de memória na qual ela está e inserindo no programa

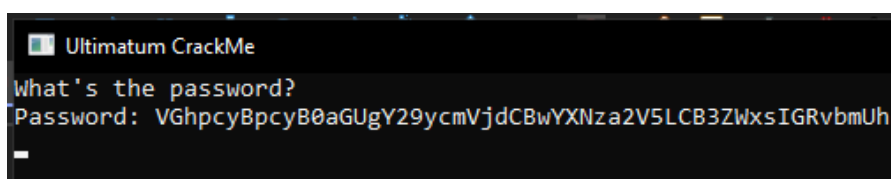


Figura 34 – Teste senha  
Fonte: Elaborada pelo autor (2023)

Seguindo toda sequência de instruções o último jne não é feito e recebemos de resposta:

```
What's the password?  
Password: VGhpcyBpcyB0aGUgY29ycmVjdCBwYXNza2V5LCB3ZWxsIGRvbmUh  
Well done! Please submit a solution on the crackmes.one website under this post!
```

Figura 35 – Senha correta  
Fonte: Elaborada pelo autor (2023)

Terminamos então de analisar o programa e achamos a senha.

Existem muitos outros programas no site de maior dificuldade. É muito interessante que você busque fazer outros exercícios para que possa fixar o conteúdo e para se acostumar com o *debugger* e a linguagem Assembly. Bons estudos!

## CONCLUSÃO

A corrupção de memória, um dos vetores de ataque mais antigos e persistentes na segurança de sistemas computacionais, permanece sendo uma preocupação preeminente em ambientes modernos. As vulnerabilidades relacionadas à corrupção de memória, como demonstrado através de nossas explorações, ocorrem quando um programa escreve dados fora dos limites apropriados de um bloco de memória alocado, permitindo a manipulação maliciosa da memória do programa para obter comportamentos indesejados.

Durante nossa discussão, exploramos falhas como overflow de buffer, que permite aos atacantes sobrescrever áreas adjacentes da memória; uso após livre, onde um programa tenta acessar um bloco de memória depois de já ter sido liberado; liberação dupla, que ocorre quando um bloco de memória é liberado mais de uma vez; ponteiro pendente, uma situação em que um ponteiro ainda aponta para uma área de memória após sua liberação; e condição de corrida, que se manifesta quando a ordem de execução de threads ou processos é mal gerenciada.

Além de entender essas vulnerabilidades, é crucial reconhecer as técnicas de exploração e os métodos práticos de prevenção. A abordagem proativa na programação, o uso de ferramentas de análise, e a aplicação de práticas de codificação

segura são medidas preventivas essenciais. Em conclusão, a corrupção de memória, apesar da evolução das técnicas de defesa, continua a ser uma ameaça significativa, exigindo vigilância e educação contínua para desenvolvedores e profissionais de segurança.

EMANIP



## REFERÊNCIA

X64DBG. An open-source x64/x32 debugger for windows. s.d. Disponível em: <<https://x64dbg.com/>>. Acesso em: 05 set. 2023

EMANIP